

Task 4: Robot Testing and Control

Solution

Button and LED functionality

- To control the LEDs and Pushbuttons, we can use several library structure for GPIO. Preferably use the GPIOzero Library because of its easy usability and faster reaction at the GPIO I/O pins.

```
1
2     from gpiozero import LED, Button
3
4     led = LED(7)
5     button = Button(5)
6
7     def PinCleanup():
8         led.close()
9         button.close()
10
11    for i in range(10):
12        if button.is_pressed:
13            led.on()
14        else:
15            led.off()
16
17    # GPIO pins to normal state
18    PinCleanup()
```

Motor Control: Rotating

- Since the robot has two motors, it means to rotate it left or right, we would need control over the speed and direction of each motor.

Apply separate PWM values to the motors.

⇒ If we set a higher PWM to the Left Motor(Motor_1), the Robot moves Right.

⇒ If we set a higher PWM to the Right Motor(Motor_2), the Robot should move Left.

- Preferably use RPi.GPIO library to implement this functionality.

- **In this Solution below, the robot circles Left**

```

1  # IN1: HIGH && IN2: LOW --> Motor1 Forward
2  def Robot_M1_FW():
3      GPIO.output(Motor1_IN1, GPIO.HIGH)
4      GPIO.output(Motor1_IN2, GPIO.LOW)
5
6
7  # IN1: HIGH && IN2: LOW --> Motor2 Forward
8  def Robot_M2_FW():
9      GPIO.output(Motor2_IN1, GPIO.HIGH)
10     GPIO.output(Motor2_IN2, GPIO.LOW)
11
12 # Set HIGHER PWM on Right Motor to move Left
13 def Circle_Mov_FWLeft():
14
15     PWM_2.ChangeDutyCycle(70)
16     Robot_M2_FW()
17     PWM_1.ChangeDutyCycle(30)
18     Robot_M1_FW()
19     time.sleep(10)
20     Robot_STOP()
21
22 while true:
23     led.on()
24     if button.is_pressed:
25         Circle_Mov_FWRight()
26     else:
27         Circle_Mov_FWLeft()

```

Motor Control: Speed Control

- By changing PWM values of the motors, one can accelerate or decelerate the Robot.
⇒ A while or for loop should be suitable.
- In this Solution below, the robot accelerates forward and then decelerates

```

1  #Speed Limit set at 80
2  def Robot_ACLT(a):
3      PWM_1.ChangeDutyCycle(30)
4      Robot_M1_FW()
5      PWM_2.ChangeDutyCycle(30)
6      Robot_M2_FW()
7      x = 30                                # Start Speed
8      while(x < 80):
9          print(x)
10         x += 10
11         PWM_1.ChangeDutyCycle(x)
12         PWM_2.ChangeDutyCycle(x)
13         time.sleep(0.5)
14
15     Robot_DCLT(x)                            # Decelerate motor
16
17 def Robot_DCLT(x):
18     PWM_1.ChangeDutyCycle(x)
19     Robot_M1_FW()
20     PWM_2.ChangeDutyCycle(x)
21     Robot_M2_FW()
22     time.sleep(0.5)
23     while(x > 10):
24         print(x)
25         x -= 10
26         PWM_1.ChangeDutyCycle(x)
27         PWM_2.ChangeDutyCycle(x)
28         time.sleep(0.5)
29     Robot_STOP()
30
31 if button.is_pressed:
32     start = time.time()
33     Robot_ACLT()    # Acceelerate motor
34     fin = time.time()
35     print(str(fin-start)+'sec')

```

Observation

- Robot moves quite a distance.
⇒ Almost 2 meters.
- To perfect the circular movement of the robot, it would be better to reduce the PWM.
- Whilst accelerating the robot forward and backward in the same path, one would notice slight shifts to the left or right. This is simply due to the small time gap before both motors run with the same PWM.
- Closing all connections is a useful practice.
It sets all GPIO pins to normal state at the end of the program.
RPi.GPIO library ⇒ `GPIO.cleanup()` .
GPIOzero library ⇒ `GPIO.close()`.